COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S4
DATE PERFORMED:	September 04, 2026	**DATE SUBMITTED:**	September 09, 2026
NAME:	Clifford Lee C. Doplayna	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 7.1
Sorting Algorithms Pt1

1. PROCEDURE OUTPUT

Code + Console Screenshot	main.cpp > main()
	1 #include <iostream>
	2 #include <cstdlib>
	3 #include <ctime>
	4 using namespace std;
	5
	6 void displayArray(int arr[], int size) {
	7 for (int i = 0; i < size; i++) {
	8 cout << arr[i] << " ";
	9 }
	10 cout << endl;
	11 }
	12
	13 int main() {
	14 const int SIZE = 100;
	15 int arr[SIZE];
	16
	17 srand(time(0));
	18
	19 for (int i = 0; i < SIZE; i++) {
	20 arr[i] = rand() % 100; // Generate random numbers between 0 and 99
	21 }
	22
	23 // Display the array
	24 cout << "100 Random Values:" << endl;
	25 displayArray(arr, SIZE);
	26
	27 return 0;
	28 }

Observations	I generated an array of 100 random integers using the rand() function. The initial array is completely unsorted, which gives me a clean starting point to test and compare each sorting algorithm.

Table 7-1. Array of Values for Sort Algorithm Testing

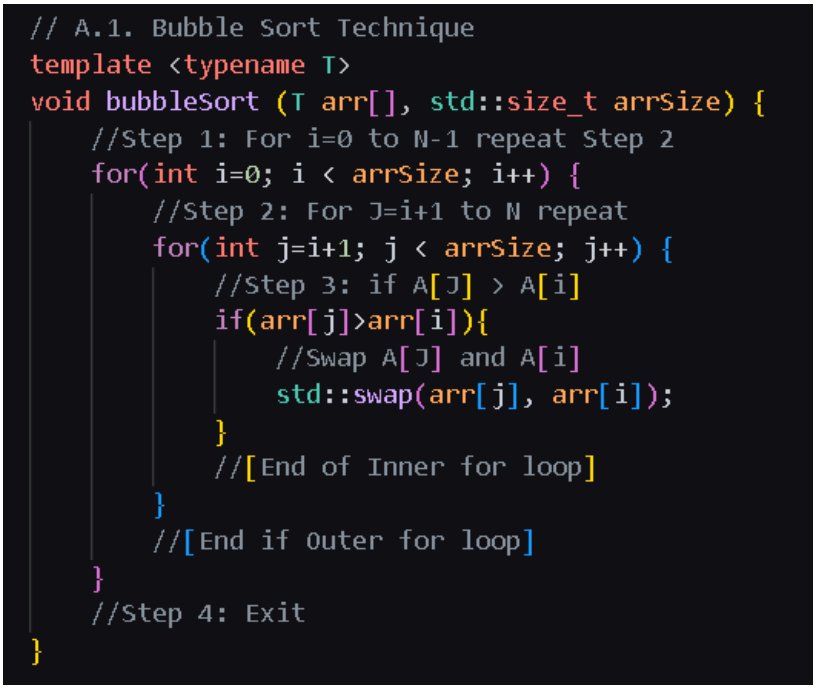
Code + Console Screenshot	
Observations	I observed that the bubble sort implementation checks adjacent pairs and swaps them if the left element is smaller. Because of this condition (arr[j] > arr[i]), the code sorts the 100 elements in descending order.

Table 7-2. Bubble Sort Technique

Code + Console Screenshot	
Observations	I noticed that this algorithm uses Routine Smallest to scan the array, find the index of the smallest value, and swap it into the current position. It systematically places the smallest remaining numbers at the front, sorting the array in ascending order.

Table 7-3. Selection Sort Algorithm

Code + Console Screenshot	

```
64  // A.3. Insertion Sort Algorithm
65  template <typename T>
66  void insertionSort (T arr[], const int N) {
67      int K=0, J, temp;
68      //Step 1: Repeat Steps 2 to 5 for k=1 to N-1
69      while (K<N) {
70          //Step 2: set temp = A[K]
71          temp = arr[K];
72          //Step 3: set J = k-1
73          J = K-1;
74          //Step 4: Repeat while temp <= A[J]
75          while (J >= 0 && temp <= arr[J]){
76              //set A[J+1] = A[J]
77              arr[J+1] = arr[J];
78              //set J = J-1
79              J--;
80          }
81          //[end of inner loop]
82          //Step 5: set A[J + 1] = temp
83          arr[J+1] = temp;
84          //[end of loop]
85          K++;
86      }
87      //Step 6: exit
88  }
89
90  #endif
```

Observations	I observed that insertion sort takes one element at a time and compares it against the already-sorted elements to its left. It shifts larger elements to the right to make room, inserting each value into its correct spot until the entire array is sorted in ascending order.

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <string>

const int TOTAL_VOTES = 100;
const int NUM_CANDIDATES = 5;

const std::string CANDIDATES[NUM_CANDIDATES] = {
    "Bo Dalton Capistrano",
    "Cornelius Raymon Agustin",
    "Deja Jayla Banaga",
    "Lalla Brielle Yabut",
    "Franklin Relano Castro"
};

void insertionSort(int arr[], int n) {
    int K = 0, J, temp;
    while (K < n) {
        temp = arr[K];
        J = K - 1;
        while (J >= 0 && temp < arr[J]) {
            arr[J + 1] = arr[J];
            J--;
        }
        arr[J + 1] = temp;
        K++;
    }
}

int main() {
    int votes[TOTAL_VOTES];
```

```
    int voteCounts[NUM_CANDIDATES] = {0};

    srand(time(0));

    for (int i = 0; i < TOTAL_VOTES; i++) {
        votes[i] = (rand() % NUM_CANDIDATES) + 1;
    }

    std::cout << "UNSORTED VOTE ARRAY" << std::endl;
    for (int i = 0; i < TOTAL_VOTES; i++) {
        std::cout << votes[i] << " ";
    }
    std::cout << "\n\n";

    insertionSort(votes, TOTAL_VOTES);

    std::cout << "SORTED VOTE ARRAY" << std::endl;
    for (int i = 0; i < TOTAL_VOTES; i++) {
        std::cout << votes[i] << " ";
    }
    std::cout << "\n\n";

    for (int i = 0; i < TOTAL_VOTES; i++) {
        voteCounts[votes[i] - 1]++;
    }

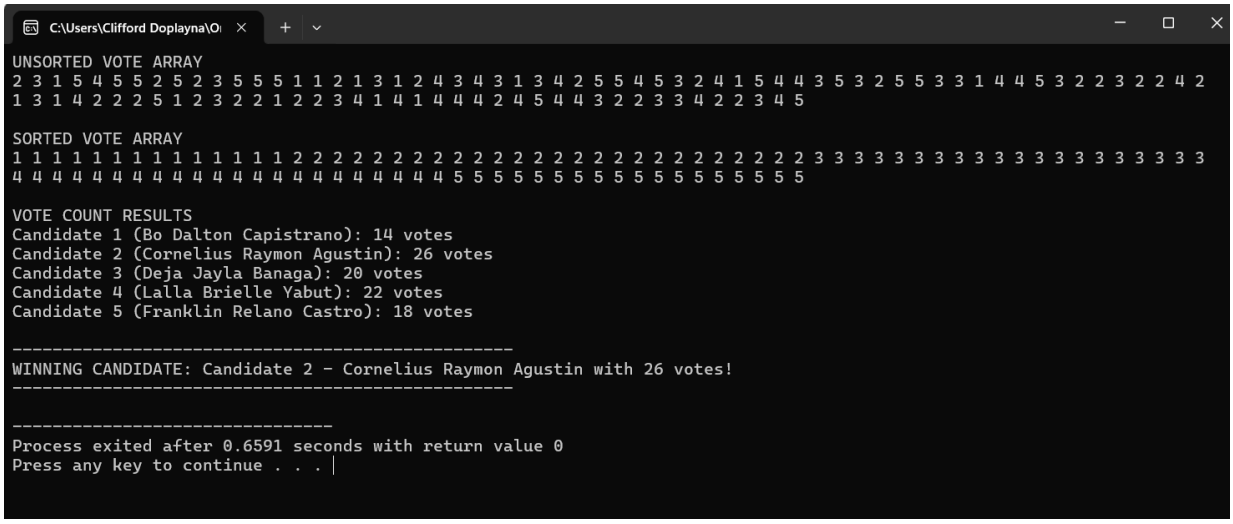
    int maxVotes = -1;
    int winningIndex = -1;

    std::cout << "VOTE COUNT RESULTS" << std::endl;
    for (int i = 0; i < NUM_CANDIDATES; i++) {
        std::cout << "Candidate " << (i + 1) << " (" << CANDIDATES[i] << "): "
                  << voteCounts[i] << " votes" << std::endl;

        if (voteCounts[i] > maxVotes) {
            maxVotes = voteCounts[i];
            winningIndex = i;
        }
    }

    std::cout << "\n--------------------------------------------------" << std::endl;
    std::cout << "WINNING CANDIDATE: Candidate " << (winningIndex + 1)
              << " - " << CANDIDATES[winningIndex]
              << " with " << maxVotes << " votes!" << std::endl;
    std::cout << "--------------------------------------------------" << std::endl;

    return 0;
}
```



3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Chatgpt,Quillbot**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Extra Information and Grammar Checker**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used the chatgpt to know more informations about sorting algorithms and how I can apply it to my code. I used quillbot for the grammar checker of my conclusion.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.

Conclusion

COMPUTER ENGINEERING

QUEZON CITY

After finishing this activity learned how basic sorting algorithms like bubble, selection, and insertion sort process and organize unsorted array elements using distinct comparisons and shifts. Moving forward. The things that I need to improve is my ability to write clean C++ header files and optimize my code to handle edge cases more efficiently.

Rubrics

T.I.P. S.O. 7 Rubric for Lifelong Learning (1)

Criteria	Ratings						Pts
Acquire and apply new knowledge from outside sources	6 pts **Excellent** Educational interests and pursuits exist and flourish outside classroom requirements, knowledge and/or experiences are pursued independently and applies knowledge learned into practice	5 pts **Good** Educational interests and pursuits exist and flourish outside classroom requirements, knowledge and/or experiences are pursued independently	4 pts **Satisfactory** Look beyond classroom requirements, showing interest in pursuing knowledge independently	3 pts **Unsatisfactory** Begins to look beyond classroom requirements, showing interest in pursuing knowledge independently	2 pts **Poor** Relies on classroom instruction only	1 pts **Very Poor** No initiative or interest in acquiring new knowledge	6 pts
Learn independently	6 pts **Excellent** Completes an assigned task independently and practices continuous improvement	5 pts **Good** Completes an assigned task without supervision or guidance	4 pts **Satisfactory** Requires minimal guidance to complete an assigned task	3 pts **Unsatisfactory** Requires detailed or step-by-step instructions to complete a task	2 pts **Poor** Shows little interest to complete a task independently	1 pts **Very Poor** No interest to complete a task independently	6 pts
Critical thinking in the broadest context of technological change	6 pts **Excellent** Synthesizes and integrates information from a variety of sources; formulates a clear and precise perspective; draws appropriate conclusions	5 pts **Good** Evaluate information from a variety of sources; formulates a clear and precise perspective	4 pts **Satisfactory** Analyze information from a variety of sources; formulates a clear and precise perspective.	3 pts **Unsatisfactory** Apply the gathered information to formulate the problem	2 pts **Poor** Gather and summarized the information from a variety of sources but failed to formulate the problem	1 pts **Very Poor** Gather information from a variety of sources	6 pts
Creativity and adaptability to new and emerging technologies	6 pts **Excellent** Ideas are combined in original and creative ways in line with the new and emerging technology trends to solve a problem or address an issue.	5 pts **Good** Ideas are creative and adapt the new knowledge to solve a problem or address an issue	4 pts **Satisfactory** Ideas are creative in solving a problem or address an issue	3 pts **Unsatisfactory** Shows some creative ways to solve the problem	2 pts **Poor** Shows initiative and attempt to develop creative ideas to solve the problem	1 pts **Very Poor** Ideas are copied or restated from the sources consulted	6 pts
						Total Points: 24	